OSSP PRACTICAL 6

Named Pipe Client–Server Communication and POSIX Signal Handling

Roll No: 2520030106

Name: D. Mohit Venkat Sai

SEC-8A

Aim

To implement a client–server application using Named Pipes (FIFOs), allow multiple clients to communicate with the server, process and return messages, and demonstrate asynchronous POSIX signal handling for SIGINT, SIGTERM and SIGUSR1.

Core outcome: the program combines two Operating Systems concepts in one executable: inter-process communication through FIFOs and asynchronous event handling through signals.

1. Problem Statement

Create a FIFO-based client–server application in which clients send messages to a server. The server identifies each client, processes its request and sends a response through a client-specific FIFO. The implementation also installs handlers for SIGINT, SIGTERM and SIGUSR1 to demonstrate asynchronous event handling.

2. Concepts Demonstrated

Concept	Implementation in the program
Named Pipe / FIFO	A server FIFO receives client requests; each client gets a private response FIFO.
Multiple clients	The parent creates multiple client processes using fork().
Client identification	Each request carries the client PID, allowing the server to select the correct response FIFO.
Signal handling	sigaction() registers one handler for SIGINT, SIGTERM and SIGUSR1.
Asynchronous event	SIGUSR1 reports an event while SIGINT/SIGTERM stop the server loop.
Process synchronization	waitpid() waits for client and server processes to finish.

3. Execution Flow

1. The program forks a server process.
2. The server creates /tmp/ossp_server_fifo and waits for requests.
3. The parent forks three client processes.
4. Each client creates /tmp/ossp_client_PID_fifo, sends PID + message to the server FIFO, and reads its response.
5. The server parses the PID, processes the message and writes a response to that client's FIFO.
6. The parent waits for clients, sends SIGUSR1 to the server, then sends SIGTERM and waits for the server to exit.

4. Important System Calls

mkfifo() — creates a named pipe. **open()/read()/write()** — transfers data through the FIFO. **fork()** — creates concurrent client/server processes. **sigaction()** — installs signal handlers. **kill()** — sends signals between processes. **waitpid()** — synchronizes process termination.

5. Source Code — Ordered by Line Sequence

The following source-code captures are arranged from the beginning of Practical_6.c to the end, rather than in the order the screenshots were supplied. This preserves the actual program flow.

Source Code — Lines 1–37: Headers, signal handler

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <signal.h>
#include <errno.h>

#define SERVER_FIFO "/tmp/ossp_server_fifo"
#define CLIENT_COUNT 3
#define BUFFER_SIZE 512

volatile sig_atomic_t running = 1;

void signal_handler(int sig)
{
    if (sig == SIGUSR1)
    {
        const char msg[] = "\nServer received SIGUSR1\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }
    else if (sig == SIGINT)
    {
        const char msg[] = "\nServer received SIGINT\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
        running = 0;
    }
    else if (sig == SIGTERM)
    {
        const char msg[] = "\nServer received SIGTERM\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
        running = 0;
    }
}
```

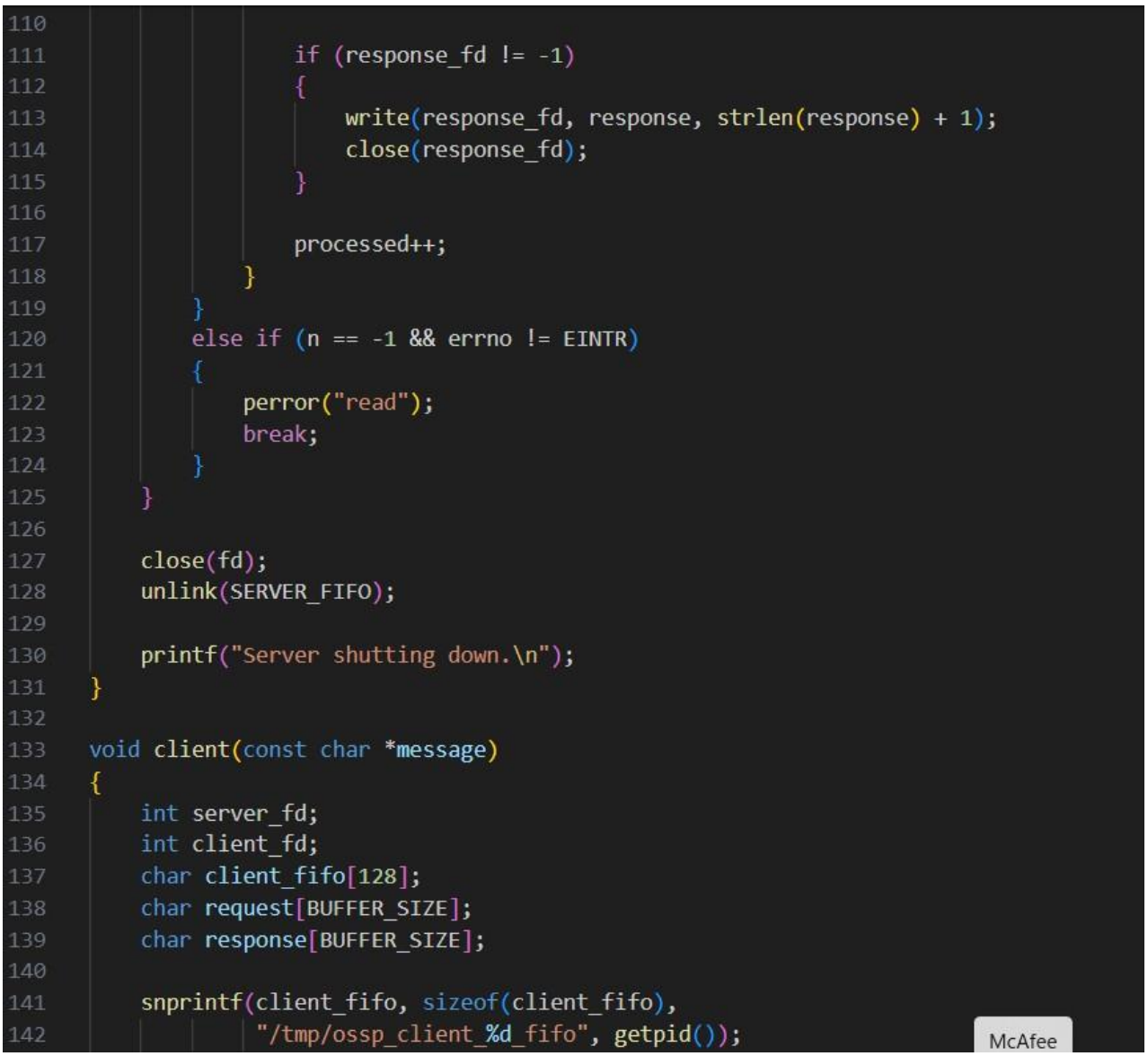
Source Code — Lines 39–75: Signal setup and FIFO server initialization

```
39  void setup_signals()
40  {
41      struct sigaction sa;
42
43      memset(&sa, 0, sizeof(sa));
44      sa.sa_handler = signal_handler;
45      sigemptyset(&sa.sa_mask);
46
47      sigaction(SIGINT, &sa, NULL);
48      sigaction(SIGTERM, &sa, NULL);
49      sigaction(SIGUSR1, &sa, NULL);
50  }
51
52  void server()
53  {
54      int fd;
55      char buffer[BUFFER_SIZE];
56      int processed = 0;
57
58      setup_signals();
59
60      unlink(SERVER_FIFO);
61
62      if (mkfifo(SERVER_FIFO, 0666) == -1 && errno != EEXIST)
63      {
64          perror("mkfifo");
65          exit(1);
66      }
67
68      printf("=== FIFO Server Started ===\n");
69      printf("Server FIFO: %s\n", SERVER_FIFO);
70
71      fd = open(SERVER_FIFO, O_RDONLY);
72      if (fd == -1)
73      {
74          perror("open");
75          exit(1);
```

Source Code — Lines 76–109: FIFO message processing

```
76      }
77
78      while (running && processed < CLIENT_COUNT)
79      {
80          ssize_t n = read(fd, buffer, sizeof(buffer) - 1);
81
82          if (n > 0)
83          {
84              char *separator;
85              char client_pid[32];
86              char client_fifo[128];
87              char response[BUFFER_SIZE];
88              int response_fd;
89
90              buffer[n] = '\0';
91
92              separator = strchr(buffer, '|');
93
94              if (separator != NULL)
95              {
96                  *separator = '\0';
97
98                  snprintf(client_pid, sizeof(client_pid), "%s", buffer);
99                  snprintf(client_fifo, sizeof(client_fifo),
100                           "/tmp/ossp_client_%s_fifo", client_pid);
101
102                 printf("Server received from Client %s: %s\n",
103                        client_pid, separator + 1);
104
105                 snprintf(response, sizeof(response),
106                          "Server response to Client %s: Message processed successfully",
107                          client_pid);
108
109                 response_fd = open(client_fifo, O_WRONLY);
110
```

Source Code — Lines 110–143: Response handling and client routine



```
110
111                 if (response_fd != -1)
112                 {
113                     write(response_fd, response, strlen(response) + 1);
114                     close(response_fd);
115                 }
116
117                 processed++;
118             }
119         }
120         else if (n == -1 && errno != EINTR)
121         {
122             perror("read");
123             break;
124         }
125     }
126
127     close(fd);
128     unlink(SERVER_FIFO);
129
130     printf("Server shutting down.\n");
131 }
132
133 void client(const char *message)
134 {
135     int server_fd;
136     int client_fd;
137     char client_fifo[128];
138     char request[BUFFER_SIZE];
139     char response[BUFFER_SIZE];
140
141     snprintf(client_fifo, sizeof(client_fifo),
142              "/tmp/ossp_client_%d_fifo", getpid());
```

McAfee

Source Code — Lines 141–180: Client FIFO communication and cleanup

```
141     snprintf(client_fifo, sizeof(client_fifo),
142              "/tmp/ossp_client_%d_fifo", getpid());
143
144     unlink(client_fifo);
145
146     if (mkfifo(client_fifo, 0666) == -1)
147     {
148         perror("client mkfifo");
149         exit(1);
150     }
151
152     server_fd = open(SERVER_FIFO, O_WRONLY);
153
154     if (server_fd == -1)
155     {
156         perror("server fifo");
157         unlink(client_fifo);
158         exit(1);
159     }
160
161     snprintf(request, sizeof(request),
162              "%d|%s", getpid(), message);
163
164     write(server_fd, request, strlen(request) + 1);
165     close(server_fd);
166
167     client_fd = open(client_fifo, O_RDONLY);
168
169     if (client_fd != -1)
170     {
171         ssize_t n = read(client_fd, response, sizeof(response) - 1);
172
173         if (n > 0)
174         {
175             response[n] = '\0';
176             printf("Client %d received: %s\n", getpid(), response);
```

Source Code — Lines 179–212: Main function and client creation

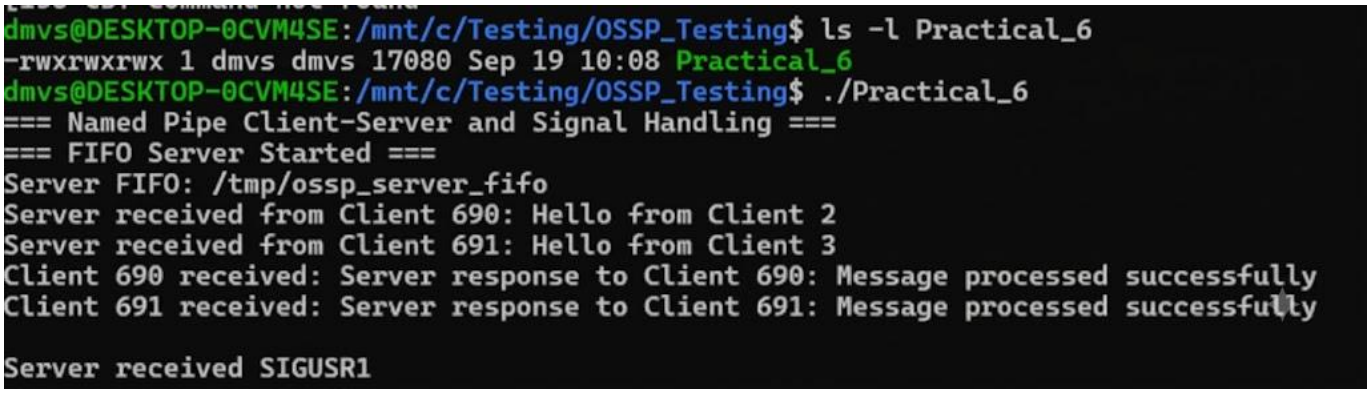
```
179         close(client_fd);
180     }
181
182     unlink(client_fifo);
183 }
184
185 int main()
186 {
187     pid_t server_pid;
188     pid_t clients[CLIENT_COUNT];
189     const char *messages[CLIENT_COUNT] = {
190         "Hello from Client 1",
191         "Hello from Client 2",
192         "Hello from Client 3"
193     };
194
195     printf("=== Named Pipe Client-Server and Signal Handling ===\n");
196
197     server_pid = fork();
198
199     if (server_pid == -1)
200     {
201         perror("fork");
202         return 1;
203     }
204
205     if (server_pid == 0)
206     {
207         server();
208         exit(0);
209     }
210
211     sleep(1);
212
213     for (int i = 0; i < CLIENT_COUNT; i++)
```

Source Code — Lines 213–245: Multiple clients, signals, wait, completion

```
213     for (int i = 0; i < CLIENT_COUNT; i++)
214     {
215         clients[i] = fork();
216
217         if (clients[i] == -1)
218         {
219             perror("fork");
220             return 1;
221         }
222
223         if (clients[i] == 0)
224         {
225             client(messages[i]);
226             exit(0);
227         }
228     }
229
230     sleep(1);
231
232     kill(server_pid, SIGUSR1);
233
234     for (int i = 0; i < CLIENT_COUNT; i++)
235         waitpid(clients[i], NULL, 0);
236
237     kill(server_pid, SIGTERM);
238
239     waitpid(server_pid, NULL, 0);
240
241     printf("\nAll clients completed.\n");
242     printf("Practical completed successfully.\n");
243
244     return 0;
245 }
```

6. Execution Output

The captured terminal output shows the FIFO server starting, receiving messages from multiple clients, returning responses, and receiving SIGUSR1. The output is reproduced from the supplied execution screenshot.



Observed behavior:

- FIFO server starts successfully.
- Messages from separate client processes are received and identified using client PIDs.
- The server sends a client-specific success response.
- SIGUSR1 is handled asynchronously and produces a server-side notification.

7. Multiple-Client FIFO Behavior

The program creates three client processes. Each client uses its own FIFO for receiving the response, while all clients write their requests to the common server FIFO. Because the requests are generated by separate processes, their arrival order is not something the program should assume in advance. The server therefore extracts the client PID from each request and routes the response to the corresponding client FIFO.

8. Role of the Signal Handler

The signal handler provides asynchronous control over the server. SIGUSR1 is treated as an informational event, whereas SIGINT and SIGTERM set the running flag to zero so the server can leave its processing loop. The handler uses write() for its immediate messages, which is appropriate for signal-handler context.

9. Signal Summary

Signal	Program behavior
SIGUSR1	Prints a notification; server continues running.
SIGINT	Prints a notification and stops the server loop.
SIGTERM	Prints a notification and stops the server loop.

10. Conclusion

The practical demonstrates two core Operating Systems mechanisms together: IPC through Named Pipes and asynchronous event handling through POSIX signals. Multiple forked clients communicate with a common FIFO server, while client-specific FIFOs provide responses. Signal handlers allow the server to react to external events and terminate in a controlled manner.

Result: The supplied program successfully demonstrates Named Pipe client–server communication, multiple-client interaction, and handling of SIGINT, SIGTERM and SIGUSR1.